PalmSource™ 99

K Virtual Machine (KVM) Java™ Technology on Palm Computing® Platform Devices

Adam Abramski
KVM Product Marketing Manager,
Consumer & Embedded, Sun Microsystems, Inc.

3Com
More connected.

Presentation Topics

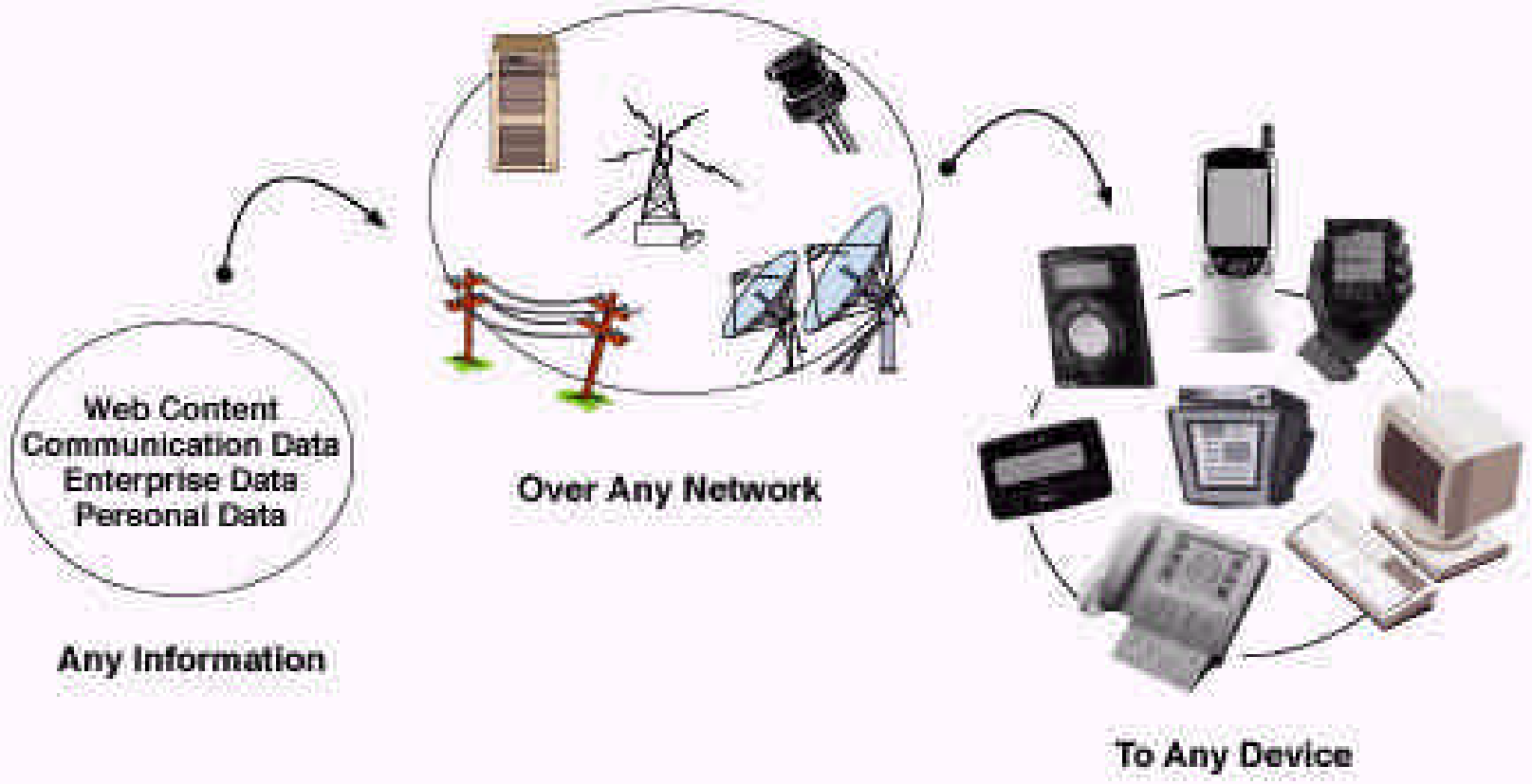
- Ubiquity of Devices and Information
- Why Java™ Technology for Consumer Electronic Devices?
- Java™ 2 Micro Edition
- VM Market Position
- What is the K Virtual Machine Technology?
- KVM Device Target Characteristics
- Roadmap
- Summary

Everything Networked

PalmSource 99



Information Available Everywhere



Why Java™ Technology for Small Platforms?

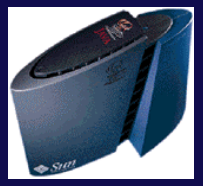
PalmSource 99

- Create completely new kinds of products
  - Support dynamic executable content for more functionality
  - Build extensible devices that support various content types
- Increase the market opportunity
  - Encourage new, highly dynamic applications
  - Integrate products into mainstream IT systems
- Reduce time-to-market
  - Ease of programming, scalability, consistency
  - Can sub-contract development easier
  - Can load applications safely after manufacture
- Reduce development costs
  - Cheaper than proprietary software tools
  - Stabilize lower layers of software
  - Unified object-based approach
  - => improve reuse; enable the future

Scaling Down Java™ Technology



16 MB





2 MB



512 kB

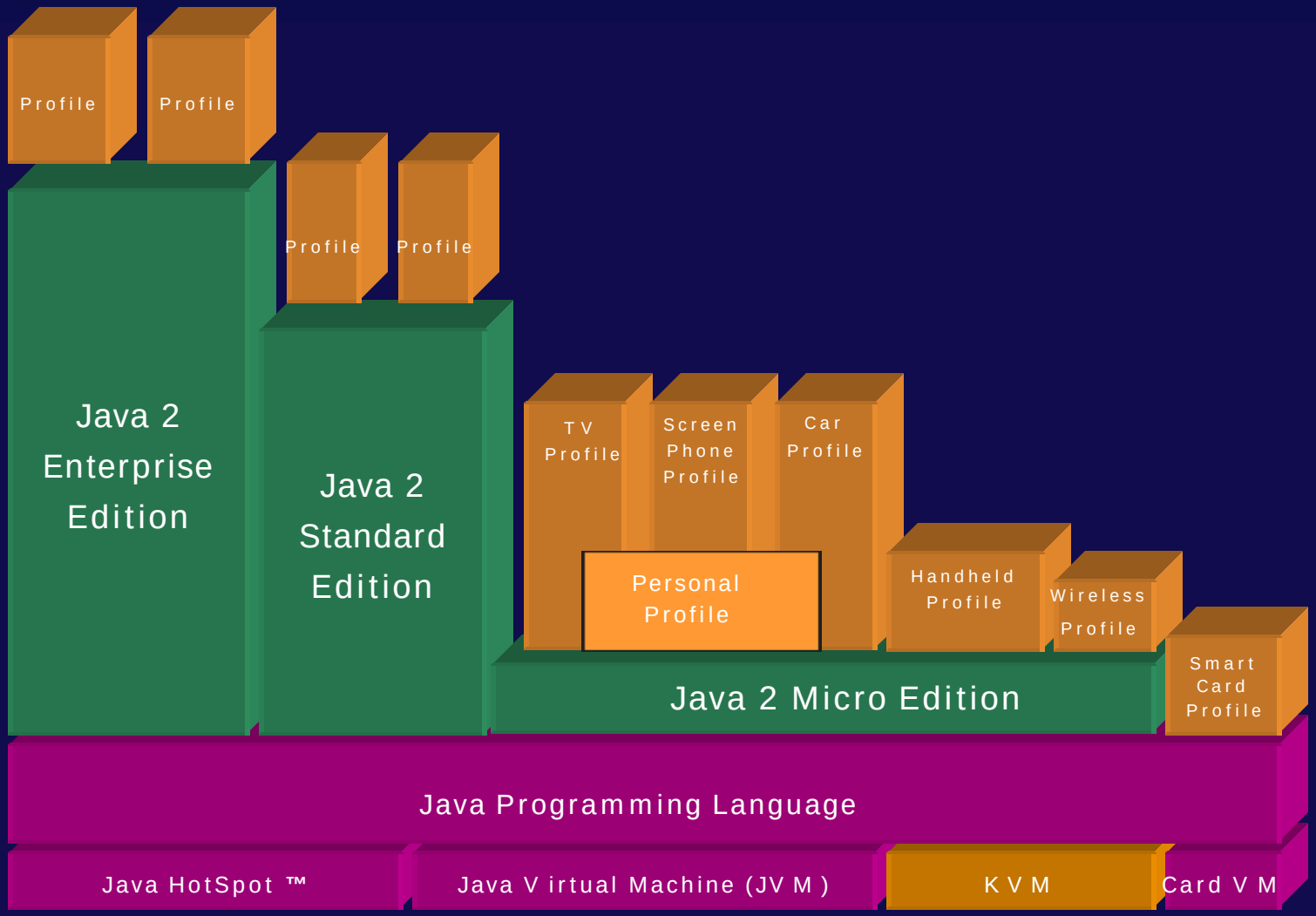


16 kB

64k/128k memory available,
not necessarily 32 bits, less than 25MHz:
Cell phones, personal organizers, pagers, EFT-POS terminals, new digital A/V devices, home appliances

Java™ 2 Platform, MicroEdition (J2ME)

PalmSource 99



VM Market Positioning



What Is the K Virtual Machine?

- Very compact Java™ Virtual Machine (JVM) implementation for consumer devices such as mobile phones, pagers, PDA's, POS terminals
- Core component of the Java™ 2 Platform, Micro Edition
- Designed for portability, ease of use and small size
- May be used in all kinds of devices which do not have enough resources to support full classic JVM

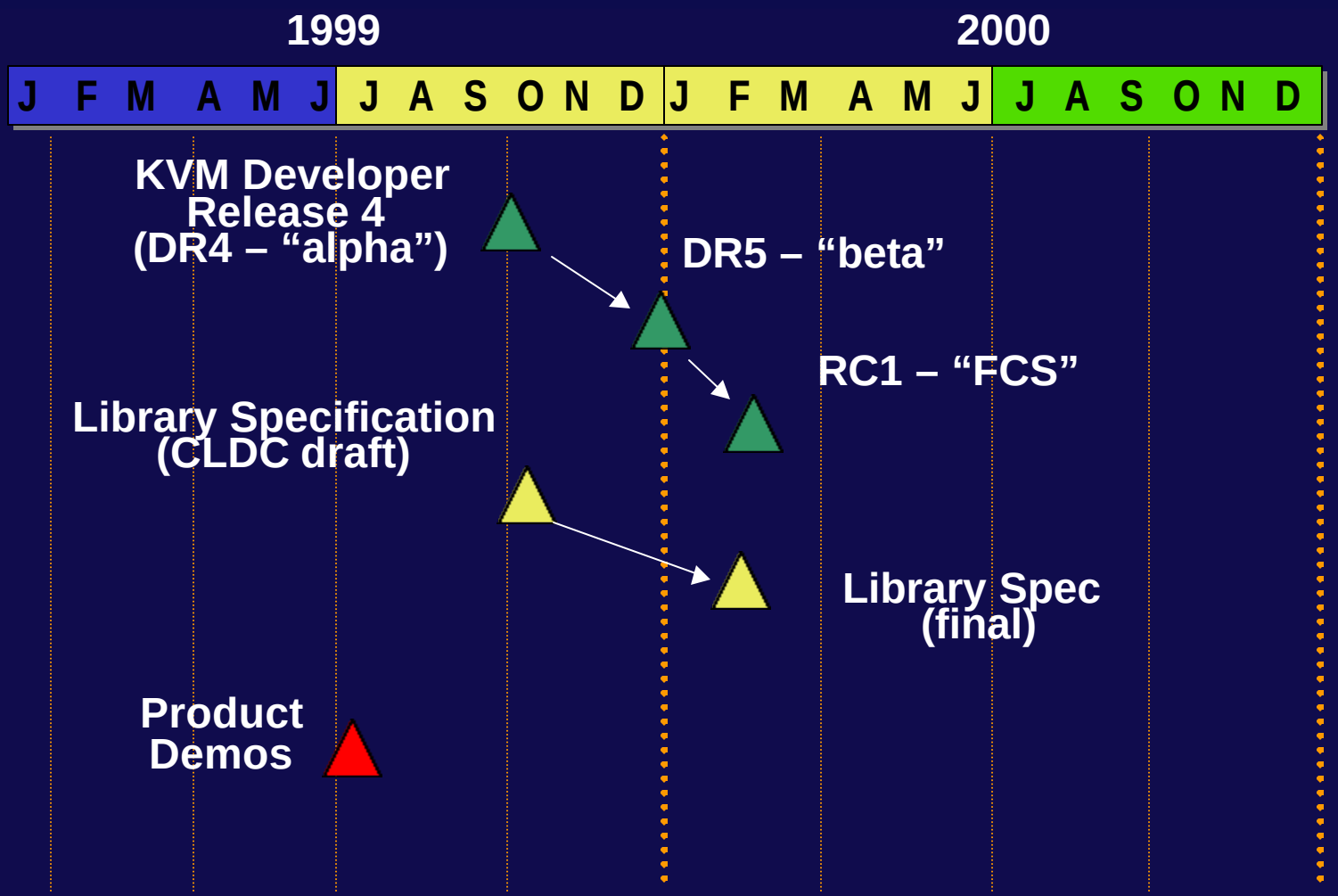
KVM Target Devices
Characteristics

PalmSource 99

- Often special purpose, limited function devices
- Limited available memory
- Limited processing power (16 bit or low-end 32 bit CPU)
- Stringent power consumption requirements (typically battery-powered)
- Network connection is low bandwidth and can be intermittent

KVM Roadmap

PalmSource 99



Summary of Sun's Approach

PalmSource 99

- Establish open standards with industry
- Specify core technology building blocks
- Assist device manufacturers
- Promote, assist, and educate ISV and content developers
- Partner to provide end-to-end solutions



Java™ Technology on Palm Computing® Platform Devices

Tasneem J. Sayeed
KVM Engineer, Consumer & Embedded,
Sun Microsystems, Inc.

3Com
More connected.

Presentation Topics

PalmSource 99

- Background & Overview
  - Design Goals & The Palm Computing® Platform
  - What is the K Virtual Machine (KVM)?
- The KVM & Java™ 2 Micro Edition (J2ME)
  - J2ME Components
  - KVM Design
- The KVM & Java 2 Micro Edition
  - Java 2 Micro Edition Application Framework
- Final Comments
  - Unsupported Features
  - Where to learn more

Background & Overview

PalmSource 99

- A gap existed between Sun's PersonalJava™ and Java Card™ Products
- Large number of consumer devices fall between desktop and smart card (cell phones, pagers, PDAs, card readers, …)
- Standard Java Virtual Machine (VM) and Java libraries were too large to run on these devices
- Java Card™ technology lacked functionality (incomplete bytecode set, no dynamic class loading, no garbage collection)

Background & Overview

PalmSource 99

- Build a small, standard Java VM optimized for size, not performance
- Make it available on a small, cheap, widely used platform that is easy to develop for
- Design Goals
  - Small size
  - Portability
  - Extensibility (must support dynamic class loading)
  - Clean source code base

The Palm Computing Platform

PalmSource 99

- 1-4 MB of Static Database Memory
  - Special API function calls needed (cannot be written directly)
  - Too slow to be used as heap memory for a VM
- 64 KB of Memory for Code
  - Multiple segments may be created if needed; rarely used
- 64-96 KB of directly addressable DRAM for data
  - >20 kB is taken by the Palm OS™; > 50 kB when networking is enabled
- Large existing user base & development tools for Palm Computing Platform readily available

What is the K Virtual Machine?

- New, compact Java Virtual Machine
  - Targeted for resource-constrained, high-volume devices
  - Typical target devices: cell phones, pagers, PDAs, mobile Point-Of-Sale (POS) terminals
  - Core component of the Java 2 Platform, Micro Edition
  - Key goal: to enable the use of the Java™ language in mass-produced appliances

KVM Technical Overview

PalmSource 99

- Standard “Red Book” Java VM implementation (except no bytecode verification)
- Some features optional depending on application profile (floats, longs, doubles, multithreading)
- Static memory footprint of the VM around 40K
- Requires 128K of total available memory
- Targeted for 16 bit/16 MHz processors or higher
- Initially developed by Sun Labs - “Spotless”

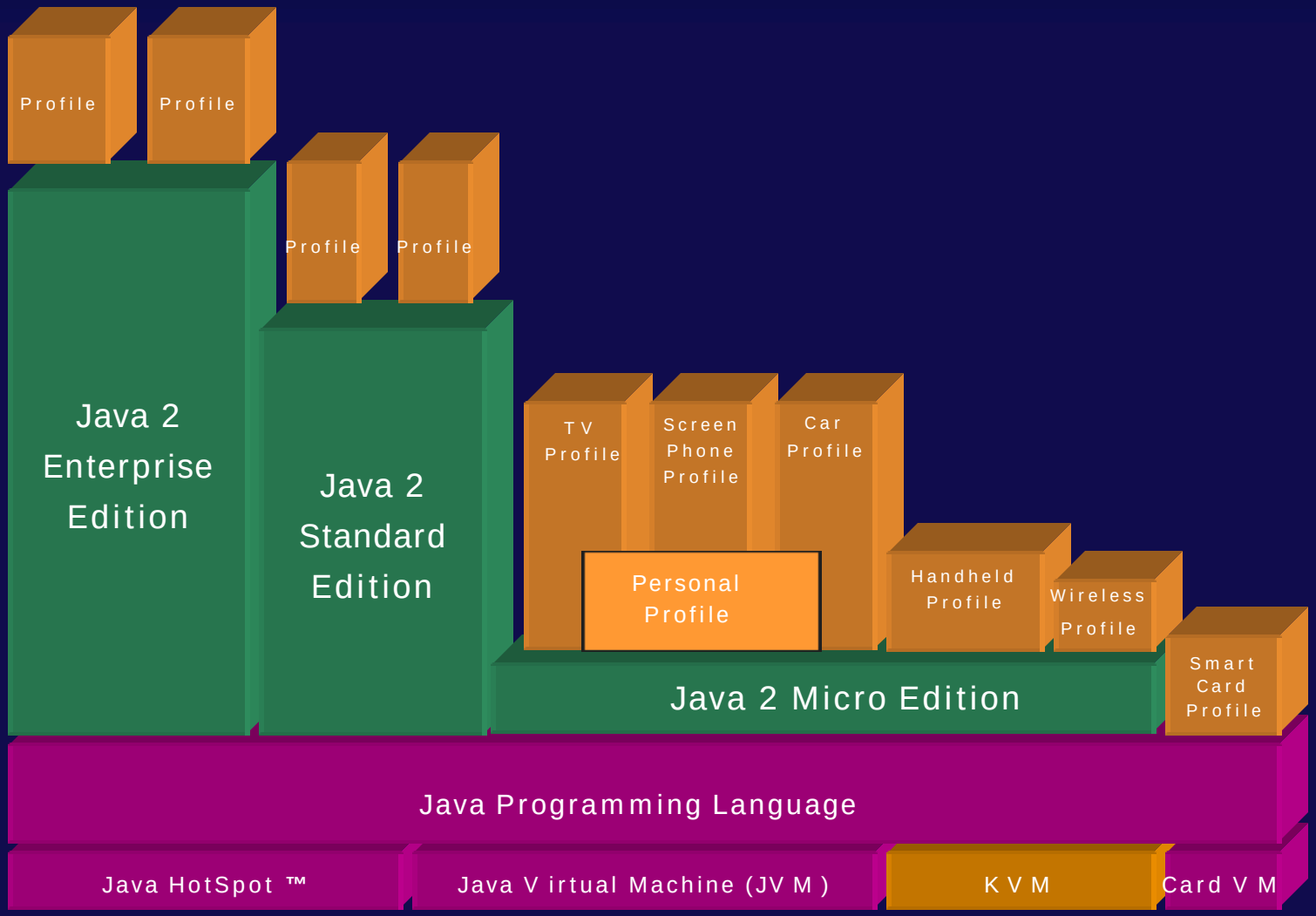
KVM Technical Overview

PalmSource 99

- Reference implementations for
  - Palm OS® Software
  - Solaris™ Operating Environment
  - Windows 95/98/NT Platforms
- Sun's customers have already ported the VM to about 20 different platforms

Java 2 Platform, MicroEdition (J2ME)

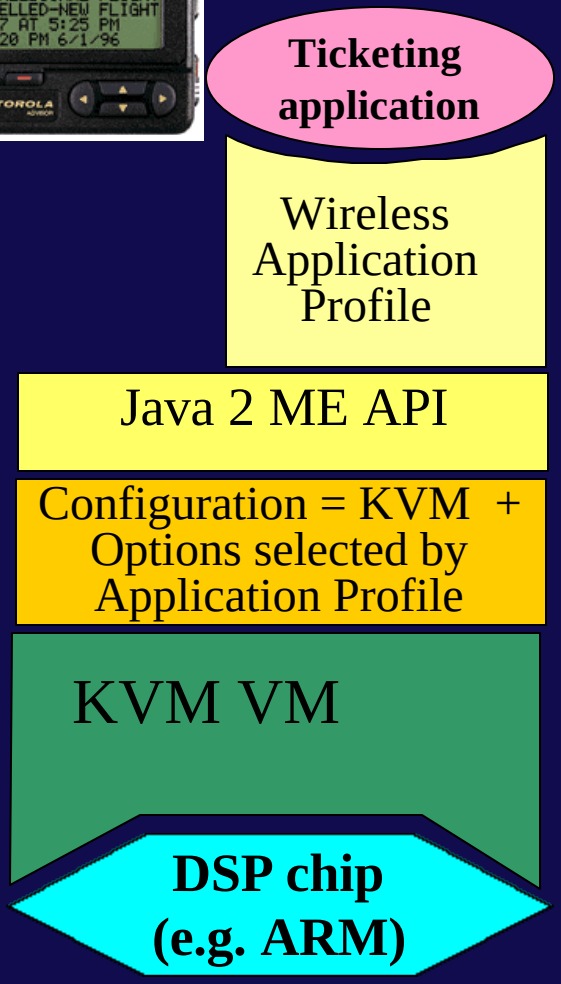
PalmSource 99



Example 1: Wireless Devices



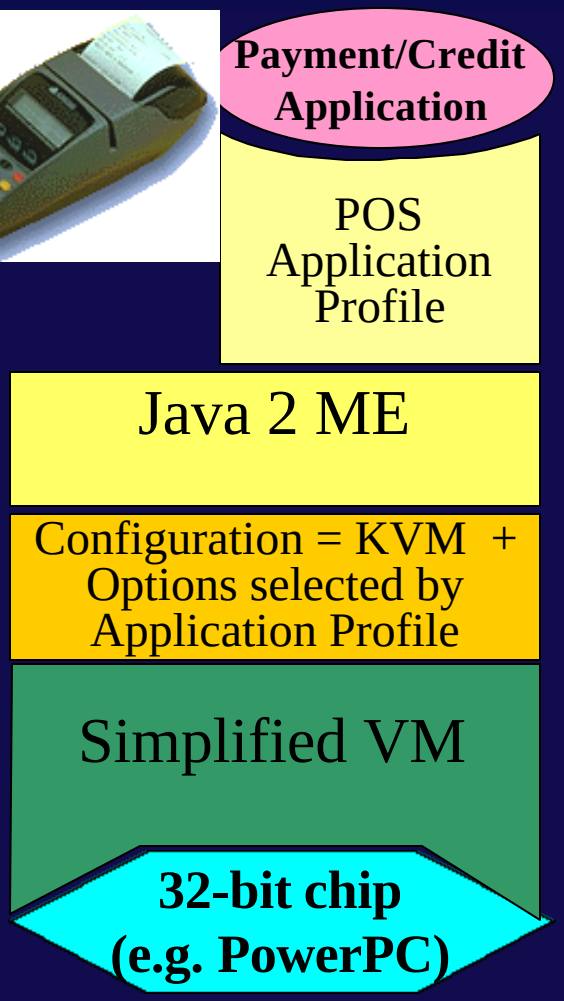




- Yellow Pages, train schedules, Games …
- Text GUI, messaging API...
- Threads, no Floats…
- 32-bit RISC, 256K ROM, 256K Flash, 64K RAM

Example 2: Payment Terminal





- ➔Payment transactions, loyalty programs
- ➔Text GUI, card interface
- ➔Dedicated Loader, no threads, no Floats…
- ➔32-bit RISC, 128K ROM, 256K Flash, 64K RAM

J2ME Components

- K Virtual Machine (KVM) based on standard Java VM
- J2ME Libraries
  - Core classes (mostly a subset of JDK)
  - Spotlet framework for Event Handling
- Classfile database builder/packager for the Palm Computing® Platform
- Demo programs

K Virtual Machine

- Implemented in C, 16,000 lines of code of which more than half are comments
  (8,000 lines of source code)
- Size of VM executable:
  45 kB on the Palm Computing® Platform
  (35 kB without the UI/event framework)
- Runtime heap size on the Palm Computing Platform:
  60 kB typical; 30 kB with networking; 4 kB min.
- Runs anywhere from 30% to 80% of the speed of JDK 1.1 without a JIT

KVM Design

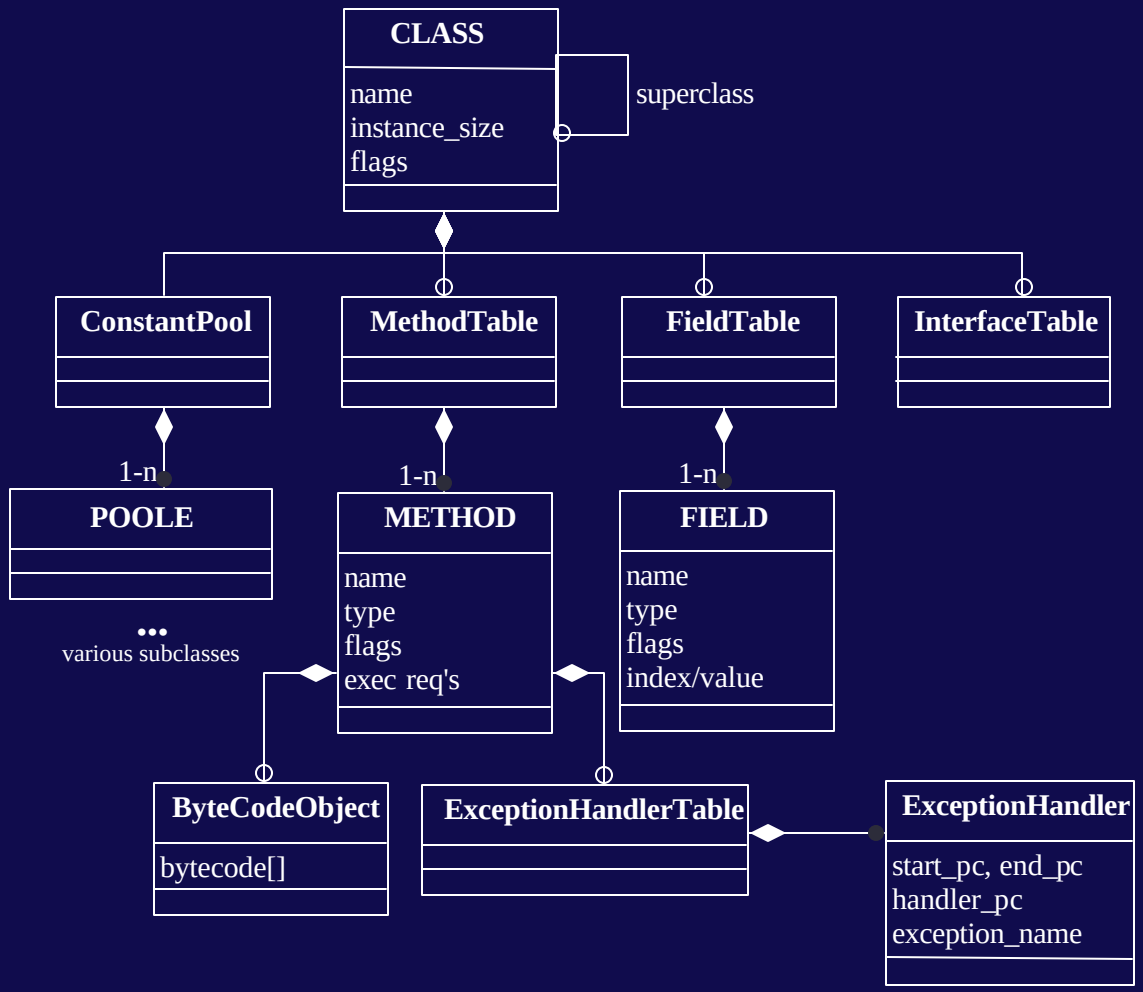
- Based on Spotless and JavaInJava virtual machines [SunLabs Tech Reports 98-64, 99-73]
- Implemented from scratch based on “The Java Virtual Machine Specification” by Lindholm and Yellin (a.k.a. the Red Book)
- Includes lots of compile time options for tuning VM (size vs. speed) and debugging
- Non-portable code minimized
- Multithreading and Garbage Collection (GC) implemented in platform-independent manner

KVM Design

- Unlike other small Java implementations,
  - KVM reads regular class files
  - Supports standard Java class loading
- Supports full Java bytecode set (basic word size is 32 bits)
- Object representation is conventional, except
  - No handles; all object references are direct (better efficiency and cleaner VM code)

Internal Class Representation

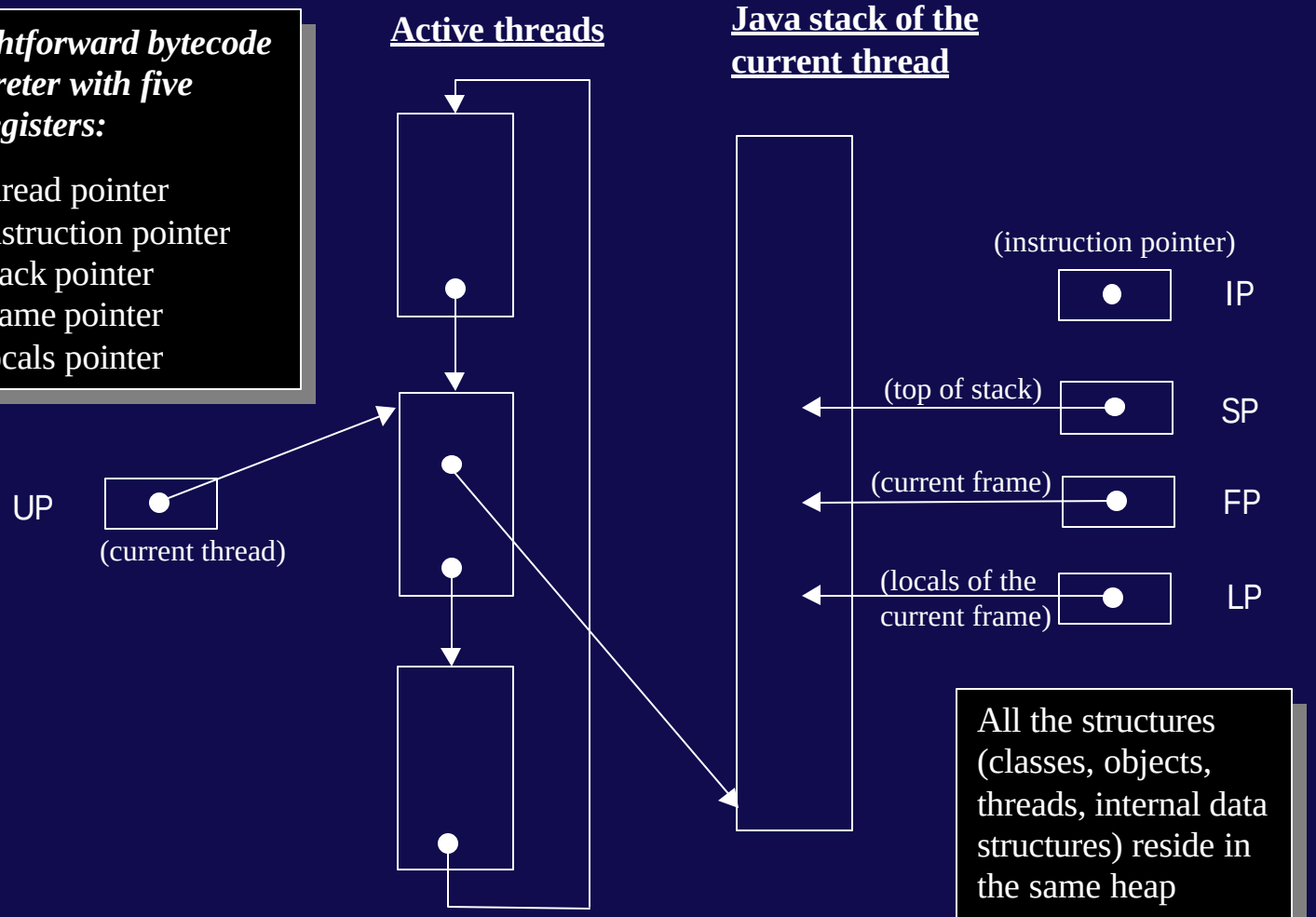
PalmSource 99



Virtual Machine Overview

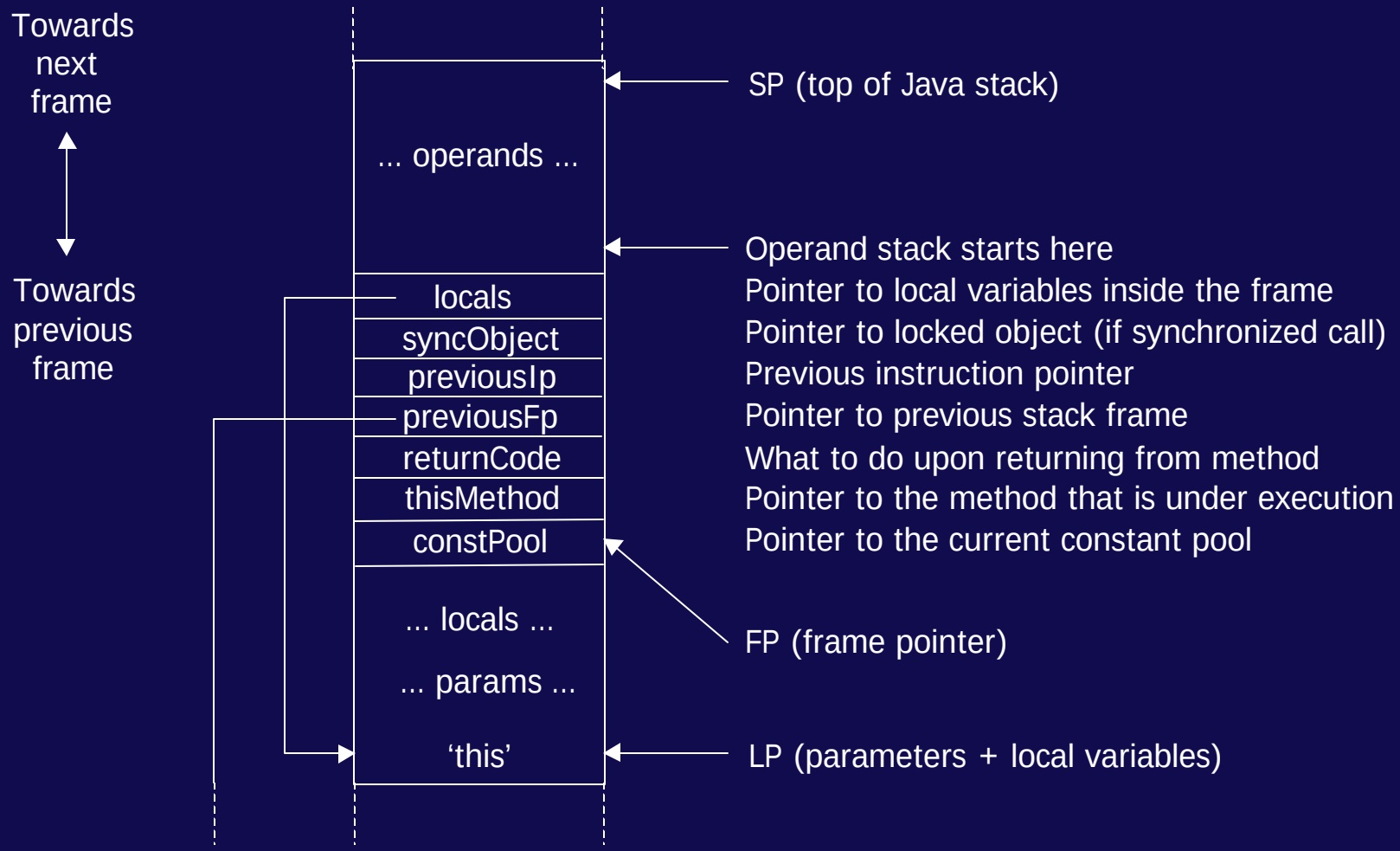
Straightforward bytecode interpreter with five VM registers:

UP thread pointer
IP instruction pointer
SP stack pointer
FP frame pointer
LP locals pointer



All the structures (classes, objects, threads, internal data structures) reside in the same heap

Stack Frame Representation



Multithreading

- KVM supports portable, fully platform-independent multithreading (green threads)
- All active threads kept in a simple linked queue, and given execution time based on priority

Multithreading

- Thread switching is fully deterministic
  - Based on number of bytecodes executed by thread
  - Makes debugging of threads easy
  - Substantially more portable & much cleaner code base than with native threads
  - Very fast thread switches (since context of thread consists of only five VM registers)
- Multithreading may be disabled easily if current device does not need multiple threads

KVM Exceptions

- Reduced runtime exceptions thrown by VM
  - NullPointerException
  - IndexOutofBoundsException
- VM now exits with error for other exceptions normally thrown by standard Java VM
- Note: the final J2ME libraries will probably have more exception classes for compatibility reasons

Garbage Collector

- Based on a non-copying collector
  - Small and simple
  - Non-moving
  - Non-incremental
  - Mark-and-Sweep algorithm
  - Single-space collector with free list
  - Optimized for small heaps (< 128-512 kB)

  - Currently working on a new, compacting collector

Optimizations

- Some performance-related optimizations supported
  - Constant pool reference caching
  - Monomorphic inline caching (not used on the Palm Computing Platform)
  - Quick bytecodes (not used on the Palm Computing Platform)
- Optimizations configurable via conditional compilation, same as debugging support
  - All internal VM structures and central operations have debug/print functions that can be included/enabled if necessary

Palm-Specific Space Optimization

- Database memory on the Palm Computing Platform is slow to write, fast to read, and there's lots of it
- Once loaded, the read-only parts of classes are moved into static database memory
  - Demo app before: 25 kB of heap space
  - Demo app after: 4 kB of heap space, 18 kB of static space

Palm-Specific Features

- Palm application launcher is used as the primary Java application manager
- Class Manager UI
  - inherited from Spotless
    - inspect and delete system classes
    - invoke VM on a selected class
    - set execution options
    - view text output

Palm-Specific Features

PalmSource 99

- Beaming
  - classfiles can be passed between devices using the IR port
  - applications can beam data to other Java applications
- Database Access
  - create databases
  - access existing Palm databases
  - read, write, modify, add, delete records

J2ME Libraries

- Implemented simple, miniaturized version of JDK with only essential classes
- Provide a simple UI framework applicable to small devices
- VM may be extended with platform-specific native functions if needed
- Libraries will go through Java Community Process (=> expect many things to change)

J2ME Libraries Overview

PalmSource 99

- Core Classes
  - java.lang.Object
  - java.lang.Class
  - java.lang.String
  - java.lang.StringBuffer
  - java.lang.Runtime
  - java.lang.System
  - java.lang.Thread
  - java.lang.Throwable
  - java.util.Bitset
  - java.util.Hashtable
  - java.util.NoSuchElementException
  - java.lang.Byte
  - java.lang.Short
  - java.lang.Integer
  - java.lang.Long
  - java.lang.Boolean
  - java.lang.Character
  - java.lang.Math
  - java.lang.Exception (…)
  - java.util.Dictionary
  - java.util.Vector

J2ME Libraries Overview

PalmSource 99

- Spotlet UI framework
  - com.sun.kjava.Spotlet
  - com.sun.kjava.Graphics
  - com.sun.kjava.Bitmap
- Database APIs
  - com.sun.kjava.Database

J2ME Application Framework

- Developed for the Palm Computing Platform
- Simple graphics and event handling functionality
- com.sun.kjava.Spotlet
  - Spotlet registers to become focus of events (`register, unregister`)
  - Includes event handler methods invoked when the associated event occurs (`penDown, penUp, penMove, keyDown`)
  - Handlers run in a dedicated thread

J2ME Application Framework

- com.sun.kjava.Spotlet
  - `public void beamReceive (byte[ ] data)`
    - used for receiving packets of data via IR from other Palm Computing Platform devices
  - `public native static boolean beamSend (byte [ ] data)`
    - used for beaming data packets via IR to another Palm Computing Platform device (explicitly called to beam data)
      Note: Other device must have registered a beamReceive handler in its current Spotlet to receive data

J2ME Application Framework

- com.sun.kjava.Graphics
  - draw shapes (`drawLine, drawRectangle, drawBorder`)
  - display strings (`drawString`)
  - get display specific properties (`getHeight, getWidth` (of a String in pixels))
  - set clipping rectangle (`setDrawRegion, resetDrawRegion`)
  - move regions (`copyRegion`)
  - draw bitmaps (`drawBitmap`)

J2ME Application Framework

- com.sun.kjava.Graphics
  - Implements a number of basic graphics operations such as line drawing, text drawing, rectangles, etc.
  - Various drawing modes supported (plain, grayscale, inverted, erase)

J2ME Application Framework

- com.sun.kjava.Database
  - `public static native boolean create(int cardno, String name, int creatorID, int typeID, boolean resDB)`
    - Creates a new database
  - `public Database(int typeID, int creatorID, int mode)`
    - Opens a database
  - `public boolean isOpen ()`
    - Checks if the database is open

J2ME Application Framework

PalmSource 99

- com.sun.kjava.Database
  - `public native byte [] getNumberOfRecords()`
    - Gets the number of records in the database
  - `public native byte [] getRecord(int recordNumber)`
    - Reads a database record into a Java byte array object
  - `public native boolean setRecord(int recordNumber, byte [] data)`
    - Sets the contents of a Palm OS database record

J2ME Application Framework

- com.sun.kjava.Database
  - `public boolean addRecord(byte [] data)`
    - Adds a new record to the end of the database
  - `public native boolean deleteRecord(int recordNumber)`
    - Deletes an existing record
  - `public native void close()`
    - `Closes the current database`

Unsupported Features

- Bytecode verification
  - `an off-device verifier may be added later`
- Java™ Native Interface (JNI)
  - too expensive for small platforms
  - native code linked directly in the VM
- Utf8, Unicode support
  - VM supports these, but the libraries are too large
- Internationalization & localization
  - VM supports these, but the libraries are too large

Unsupported Features

- Standard Java graphics support (AWT)
- Reflection, RMI and other advanced Java 2 Platform features
  - needed for JINI™, but size is an issue

More Information...

- `http: //java.sun.com/j2me`
- `http://java.sun.com/products/kvm`
- `http://www.sun.com/consumer-embedded/cover/j2me.html`

- KVM Technology Users Mailing List
  - To subscribe, send email to `listserv@java.sun.com`
  - In the body of the message, type `subscribe KVM-INTEREST`


PalmSource™ 99
3Com
More connected.